Door Camera Face Recognition System

A complete, step-by-step build guide for an AI Engineer portfolio project: 2 cameras, 1 old laptop, no Home Assistant, every component coded by you.

What the finished system does
Reads 2 camera streams all day, records them, and detects people and vehicles at the door.
Recognises family (owners), other relatives, the dustman, neighbours, and family cars.
Sends you a Telegram message with a photo when someone knocks, enters, or is unknown.
Logs every event in a database and shows them on a small web dashboard.
Trains its own recogniser on a custom dataset built from your old camera footage (Part 2).

How to use this guide. Work through the chapters in order. Each chapter ends with a box called *What you do now* that tells you exactly what to run or write, and what result proves it works. Do not move to the next chapter until the check passes. All code is a starting point: type it yourself, understand each line, then improve it. That is what makes it a real portfolio project.

Contents

0. Before you start: goals, legal and privacy
1. Architecture and folder structure
2. Setting up the laptop and Python environment
3. Connecting to the cameras (RTSP)
4. 24-hour recording with FFmpeg
5. Motion detection (the CPU saver)
6. Detecting people and cars with YOLO
7. Face detection and embeddings (InsightFace)
8. The identity database and enrollment
PART 2 - TRAINING: T1. Training strategy and data plan
T2. Organise old footage, find useful segments
T3. Extract aligned face crops
T4. Label fast with embeddings + clustering
T5. Final dataset, video-based splits, data card
T6. Training A: classifier on frozen embeddings
T7. Training B: ArcFace adapter (PyTorch)
T8. Training C (optional): fine-tune the face network
T9. Train the family-car recogniser
T10. Final evaluation, integration, retraining loop
9. Matching, thresholds and the unknown class
10. Tracking and multi-frame voting
11. Recognising the family cars
12. Event logic: knocking, entering, rules
13. Telegram notifications
14. The main pipeline (putting it all together)
15. Dashboard with FastAPI
16. Performance tuning for an old laptop
17. Evaluating accuracy like an engineer
18. Running 24/7 as a service
19. Packaging the project for your CV
20. 11-week schedule
21. Troubleshooting table
22. Resources and books

0. Before you start

0.1 Project goals (write these in your README)

- **G1** - Continuous ingestion of 2 RTSP camera streams with automatic reconnection.
- **G2** - Identify known people by face and known cars by plate or appearance.
- **G3** - Treat everyone unrecognised as *unknown* and alert immediately.
- **G4** - Run in near real time on CPU only (old laptop).
- **G5** - Measured accuracy: report false accept and false reject rates on your own test set.

0.2 Legal and privacy (important and good for your CV)

Face recognition of people is regulated in many countries. A safe approach that also impresses interviewers:

- Get **consent** from every person you enrol (family, neighbours, dustman). Tell them what is stored and why.
- Keep everything **local**: no cloud upload of faces. Store embeddings (numbers), not extra copies of photos, when possible.
- Keep recordings and snapshots for a limited time (for example 14 days) and delete automatically.
- Protect your Telegram token, camera passwords and database. Never commit them to GitHub.
- Check the law in your country about cameras that film public areas such as the street or neighbours' doors.

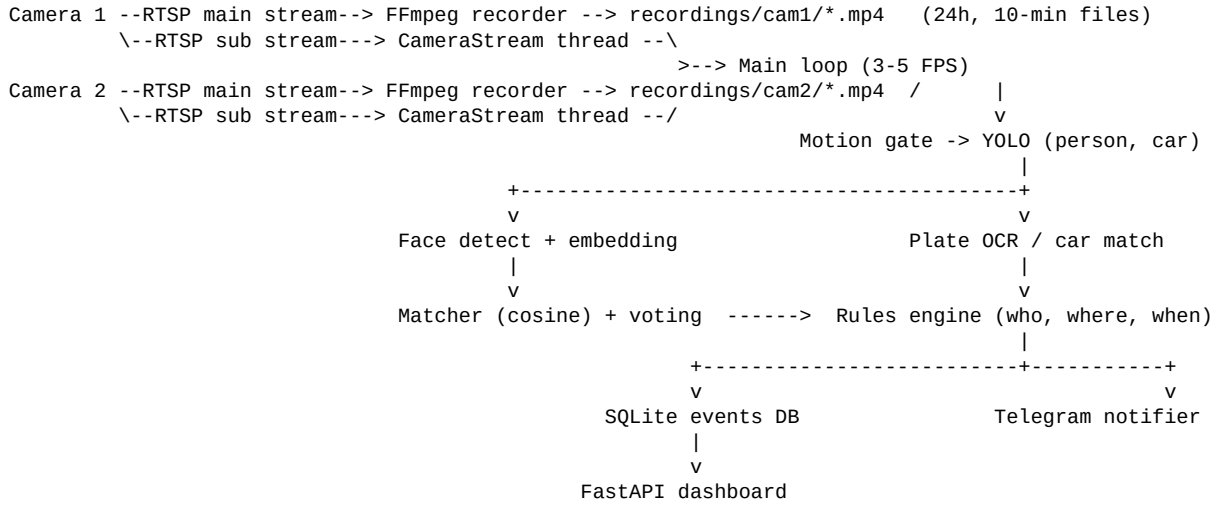
Note: The pretrained InsightFace models are released for non-commercial research use, and Ultralytics YOLO uses the AGPL licence. That is fine for a personal portfolio project. Mention it in your README.

0.3 What you need

Item	Requirement
Laptop	Any 64-bit laptop with 4 GB RAM or more, ideally 8 GB. Intel i3/i5 from 2015+ is enough at 3-5 FPS.
Operating system	Ubuntu 22.04/24.04 is strongly recommended (easier installs, runs for months). Windows works but is harder.
Cameras	IP cameras with RTSP support (almost all Hikvision, Dahua, TP-Link Tapo, Reolink, etc.). Analog cameras need a DVR or an encoder.
Network	Cameras and laptop on the same router. Use a cable for the laptop if possible.
Storage	A 500 GB+ disk. 2 cameras at 4 Mbps use about 80 GB per day, so plan retention.
Phone	Telegram app (free) for notifications.

1. Architecture and folder structure

The system is a pipeline. Each camera has its own thread that only reads frames. One main loop picks the newest frame at a low rate, checks for motion, detects people and vehicles, recognises faces, applies rules and sends alerts. Recording is a separate FFmpeg process so that an AI crash never stops your recordings.



Folder structure to create

```
door_ai/
  config.yaml            # camera URLs, zones, thresholds, rules
  .env                   # secrets: TELEGRAM_TOKEN, TELEGRAM_CHAT_ID  (never commit)
  requirements.txt
  dataset/               # enrollment photos: dataset/family__Ahmed/1.jpg ...
  recordings/            # cam1/, cam2/ (created by FFmpeg)
  snapshots/             # event images
  db/door.sqlite
  src/
    camera.py            # CameraStream (threaded, auto-reconnect)
    motion.py            # MotionDetector
    detector.py          # YOLO wrapper
    faces.py             # InsightFace wrapper
    database.py          # SQLite helpers
    enroll.py            # builds embeddings from dataset/
    matcher.py           # FaceMatcher + IdentityVoter
    tracker.py           # CentroidTracker
    plates.py            # car recognition
    rules.py             # event logic
    notifier.py          # Telegram
    main.py              # the pipeline
    dashboard.py         # FastAPI app
  tests/
  eval/                  # evaluation scripts and results
```

What you do now: Create this folder tree now (mkdir -p door_ai/src door_ai/dataset ...) and run `git init` inside door_ai. Add a .gitignore containing .env, recordings/, snapshots/, dataset/ and db/.

2. Setting up the laptop and Python

1. Install Ubuntu 22.04 or 24.04 (or use what you have). Update: `sudo apt update && sudo apt upgrade -y`
2. Install tools: `sudo apt install -y python3-venv python3-pip ffmpeg git sqlite3 vlc`
3. Create the environment: `cd door_ai && python3 -m venv .venv && source .venv/bin/activate`
4. Install the CPU-only PyTorch first (smaller, avoids huge CUDA downloads), then the rest.

```
pip install --upgrade pip
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install opencv-python numpy ultralytics insightface onnxruntime easyocr \
            requests python-dotenv pyyaml fastapi uvicorn matplotlib scikit-learn
pip freeze > requirements.txt
```

Test that everything imports:

```
python -c "import cv2, numpy, onnxruntime, insightface, ultralytics, easyocr; print('OK', \
    cv2.__version__)"
```

Note: If insightface fails to build, install build tools: sudo apt install -y build-essential python3-dev. On Windows you need the 'Microsoft C++ Build Tools'. If installs keep failing, use Ubuntu - it saves days.

What you do now: Run the import test. You must see 'OK' and a version number before continuing.

3. Connecting to the cameras (RTSP)

3.1 Find each camera's IP address

- Open your router admin page (often 192.168.1.1) and look at the connected devices list, or use the camera's phone app.
- In the router, set a **DHCP reservation** (fixed IP) for each camera so the address never changes.
- Create a separate camera user in the camera settings (for example 'aiuser') with a strong password and view-only rights.

3.2 Build and test the RTSP URL

Every brand uses a different path. Check your camera manual. Common patterns (verify for your model):

Brand	Main stream (high-res)	Sub stream (low-res, use for AI)
Hikvision	rtsp://user:pass@IP:554/Streaming/Channels/101	rtsp://user:pass@IP:554/Streaming/Channels/102
Dahua	rtsp://user:pass@IP:554/cam/realmonitor?channel=1&subtype=0	rtsp://user:pass@IP:554/cam/realmonitor?channel=1&subtype=1
TP-Link Tapo	rtsp://user:pass@IP:554/stream1	rtsp://user:pass@IP:554/stream2
Reolink	rtsp://user:pass@IP:554/h264Preview_01_main	rtsp://user:pass@IP:554/h264Preview_01_sub

Test in VLC first: Media > Open Network Stream > paste the URL. If VLC shows video, your URL is right. If your password has special characters (@, #, :), replace them with percent codes (@ = %40, # = %23).

3.3 Why use two streams per camera

- The **main stream** (1080p or more) is recorded with FFmpeg without re-encoding (low CPU).
- The **sub stream** (640x360 or 640x480) is read by Python for AI. Smaller frames mean faster processing.
- Many cameras limit simultaneous connections; using two streams spreads the load.

Note: Faces are small in a low-resolution stream. If recognition is poor, try the sub stream at 1280x720 (set it in the camera settings), or run AI on the main stream at 1-2 FPS only when motion is detected.

3.4 config.yaml

```
cameras:
  - name: door_outside
    rtsp_main: "rtsp://aiuser:PASS@192.168.1.50:554/Streaming/Channels/101"
    rtsp_sub:  "rtsp://aiuser:PASS@192.168.1.50:554/Streaming/Channels/102"
    entry_line: [[0, 300], [640, 300]]     # line in sub-stream pixels, see chapter 12
  - name: door_inside
    rtsp_main: "rtsp://aiuser:PASS@192.168.1.51:554/stream1"
    rtsp_sub:  "rtsp://aiuser:PASS@192.168.1.51:554/stream2"
    entry_line: [[0, 280], [640, 280]]

processing:
  fps: 4
  yolo_size: 480
  face_threshold: 0.45
  votes_needed: 3
  unknown_after_seconds: 6
  alert_cooldown_seconds: 90
```

3.5 The threaded camera reader (src/camera.py)

Reading frames inside your main loop is a classic beginner bug: OpenCV buffers frames, so you end up analysing old images with a growing delay. The fix is one thread per camera that always reads and keeps only the newest frame.

```
import cv2, threading, time

class CameraStream:
    def __init__(self, name, url):
        self.name, self.url = name, url
        self.frame = None
        self.last_ok = 0
        self.lock = threading.Lock()
        self.running = True
        self.thread = threading.Thread(target=self._loop, daemon=True)
        self.thread.start()

    def _open(self):
        cap = cv2.VideoCapture(self.url, cv2.CAP_FFMPEG)
        cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
        return cap

    def _loop(self):
        cap = self._open()
        while self.running:
            ok, frame = cap.read()
            if not ok:                      # stream dropped: reconnect
                cap.release()
                time.sleep(3)
                cap = self._open()
                continue
            with self.lock:
                self.frame = frame
                self.last_ok = time.time()
        cap.release()

    def read(self):
        with self.lock:
```

```
            return None if self.frame is None else self.frame.copy()

    def is_alive(self):
        return time.time() - self.last_ok < 10

    def stop(self):
        self.running = False
```

Quick test (src/test_camera.py):

```
import cv2, yaml, time
from camera import CameraStream
cfg = yaml.safe_load(open("../config.yaml"))
cams = [CameraStream(c["name"], c["rtsp_sub"]) for c in cfg["cameras"]]
time.sleep(3)
while True:
    for c in cams:
        f = c.read()
        if f is not None:
            cv2.imshow(c.name, f)
    if cv2.waitKey(30) == 27:        # ESC to quit
        break
```

What you do now: Run test_camera.py. You must see two live windows. Unplug a camera for 10 seconds and plug it back: the window should recover by itself.

4. 24-hour recording with FFmpeg

FFmpeg copies the stream into files without decoding it, so it uses almost no CPU. Files are cut every 10 minutes, which makes them easy to find and delete.

```
mkdir -p recordings/door_outside
ffmpeg -rtsp_transport tcp -i "rtsp://aiuser:PASS@192.168.1.50:554/Streaming/Channels/101" \
  -c copy -f segment -segment_time 600 -reset_timestamps 1 -strftime 1 \
  "recordings/door_outside/%Y-%m-%d_%H-%M-%S.mp4"
```

4.1 Keep it running (auto-restart)

Write a small Python supervisor (src/recorder.py) that starts one FFmpeg process per camera and restarts it if it stops:

```
import subprocess, time, yaml, os

cfg = yaml.safe_load(open("config.yaml"))
procs = {}

def start(cam):
    os.makedirs(f"recordings/{cam['name']}", exist_ok=True)
    cmd = ["ffmpeg", "-loglevel", "error", "-rtsp_transport", "tcp", "-i", cam["rtsp_main"],
           "-c", "copy", "-f", "segment", "-segment_time", "600", "-reset_timestamps", "1",
           "-strftime", "1", f"recordings/{cam['name']}/%Y-%m-%d_%H-%M-%S.mp4"]
    return subprocess.Popen(cmd)

while True:
    for cam in cfg["cameras"]:
        p = procs.get(cam["name"])
        if p is None or p.poll() is not None:
            procs[cam["name"]] = start(cam)
    time.sleep(5)
```

4.2 Automatic cleanup so the disk never fills

```
# src/cleanup.py  - run daily from cron:  0 3 * * * /path/.venv/bin/python /path/src/cleanup.py
import os, time
KEEP_DAYS = 10
now = time.time()
for root, _, files in os.walk("recordings"):
    for f in files:
        path = os.path.join(root, f)
        if now - os.path.getmtime(path) > KEEP_DAYS * 86400:
            os.remove(path)
for root, _, files in os.walk("snapshots"):
    for f in files:
        path = os.path.join(root, f)
        if now - os.path.getmtime(path) > 30 * 86400:   # snapshots kept 30 days
            os.remove(path)
```

What you do now: Start recorder.py, wait 12 minutes, and open a recorded .mp4 in VLC. Calculate: (GB per day) x KEEP_DAYS must be smaller than your free disk.

5. Motion detection (the CPU saver)

Running neural networks on every frame of an empty doorway wastes 95 percent of your CPU. A cheap motion detector acts as a gate: the AI only wakes up when pixels change. This idea (cascade of cheap-then-expensive models) is a good talking point in interviews.

```
# src/motion.py
import cv2

class MotionDetector:
    def __init__(self, min_area=1200):
        self.bg = cv2.createBackgroundSubtractorMOG2(history=300, varThreshold=32,
                detectShadows=False)
        self.min_area = min_area

    def detect(self, frame):
        small = cv2.resize(frame, None, fx=0.5, fy=0.5)
        mask = self.bg.apply(small)
        mask = cv2.medianBlur(mask, 5)
        mask = cv2.dilate(mask, None, iterations=2)
        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        return any(cv2.contourArea(c) > self.min_area for c in contours)
```

Tuning

- Too many false triggers (trees, shadows, headlights): increase **min_area** or **varThreshold**.
- Missing people: decrease them.
- Keep the detector running for 3-5 seconds *after* the last motion so slow-walking people are not lost (add a timer in main.py).

What you do now: Write a test that shows the camera with 'MOTION' printed on the image when detect() is True. Walk in front of the camera and outside it to tune min_area.

6. Detecting people and cars with YOLO

YOLOv8-nano is a small, fast object detector. In the COCO dataset the class IDs we need are: 0 = person, 2 = car, 5 = bus, 7 = truck.

```
# src/detector.py
from ultralytics import YOLO

PERSON, VEHICLES = 0, {2, 3, 5, 7}      # car, motorcycle, bus, truck

class ObjectDetector:
    def __init__(self, size=480, conf=0.4):
        self.model = YOLO("yolov8n.pt")  # downloads on first run (needs internet once)
        self.size, self.conf = size, conf

    def detect(self, frame):
        res = self.model(frame, imgsz=self.size, conf=self.conf,
                         classes=[0, 2, 3, 5, 7], verbose=False)[0]
        out = []
        for b in res.boxes:
            cls = int(b.cls[0])
            x1, y1, x2, y2 = map(int, b.xyxy[0])
            out.append({"cls": cls, "conf": float(b.conf[0]), "box": (x1, y1, x2, y2)})
        return out

    def persons(self, dets):
        return [d for d in dets if d["cls"] == PERSON]

    def vehicles(self, dets):
        return [d for d in dets if d["cls"] in VEHICLES]
```

Optional speed-up: export to ONNX once and use the exported file. It is often 1.5x to 2x faster on CPU.

```
python -c "from ultralytics import YOLO; YOLO('yolov8n.pt').export(format='onnx', imgsz=480)"
# then: YOLO('yolov8n.onnx')
```

What you do now: Draw boxes on the live frame and print the time per detection. On an old i5 you should see roughly 80-200 ms. If it is above 400 ms, lower yolo_size to 384 or 320.

7. Face detection and embeddings (InsightFace)

7.1 The idea you must understand

A face recognition system has two models. A **detector** finds faces in the image (box + 5 landmarks). A **recognition network** (ArcFace) converts an aligned face into an **embedding**: a vector of 512 numbers. Photos of the same person give vectors that point in similar directions; different people give different directions. We measure this with **cosine similarity** (1.0 = identical, 0 = unrelated). Recognition is therefore just: embed the new face, compare with the stored embeddings, choose the best match if it is above a threshold.

```
# src/faces.py
import numpy as np
from insightface.app import FaceAnalysis

class FaceEngine:
    def __init__(self, det_size=(320, 320)):
        # buffalo_sc = small & fast pack (SCRFD-500M detector + MobileFaceNet). Use buffalo_l for
        # accuracy.
        self.app = FaceAnalysis(name="buffalo_sc", providers=["CPUExecutionProvider"])
        self.app.prepare(ctx_id=-1, det_size=det_size)

    def analyse(self, image_bgr):
        faces = self.app.get(image_bgr)
        out = []
        for f in faces:
            x1, y1, x2, y2 = f.bbox.astype(int)
```

```
            out.append({"box": (x1, y1, x2, y2),
                        "det_score": float(f.det_score),
                        # length 512, unit norm
                        "embedding": f.normed_embedding.astype(np.float32),
                        "size": min(x2 - x1, y2 - y1)})
        return out
```

7.2 Quality filtering (this step separates good projects from average ones)

- Ignore faces smaller than about 60 pixels (embedding is unreliable).
- Ignore faces with det_score below 0.6.
- Ignore blurry faces: compute the variance of the Laplacian of the face crop and skip if it is below roughly 40 (tune it).
- Prefer frontal faces. People walking away or looking down will stay 'unknown' until a better frame appears - that is correct behaviour.

```
import cv2
def sharpness(crop):
    return cv2.Laplacian(cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY), cv2.CV_64F).var()

def good_face(frame, f, min_size=60, min_det=0.6, min_sharp=40):
    x1, y1, x2, y2 = f["box"]
    crop = frame[max(0, y1):y2, max(0, x1):x2]
    if crop.size == 0:
        return False
    return f["size"] >= min_size and f["det_score"] >= min_det and sharpness(crop) >= min_sharp
```

What you do now: Run FaceEngine on a photo of yourself and print the embedding shape (512,). Compare two photos of you and one photo of someone else with np.dot(a, b). Expect roughly 0.5-0.8 for you vs you and below 0.3 for different people.

8. The identity database and enrollment

8.1 SQLite schema (src/database.py)

```
import sqlite3, numpy as np, datetime

def connect(path="db/door.sqlite"):
    con = sqlite3.connect(path, check_same_thread=False)
    con.executescript("""
    CREATE TABLE IF NOT EXISTS people(
        -- owner, family, dustman, neighbor
        id INTEGER PRIMARY KEY, name TEXT UNIQUE, category TEXT);
    CREATE TABLE IF NOT EXISTS face_embeddings(
        id INTEGER PRIMARY KEY, person_id INTEGER, embedding BLOB, source TEXT);
    CREATE TABLE IF NOT EXISTS vehicles(
        id INTEGER PRIMARY KEY, plate TEXT UNIQUE, owner TEXT, category TEXT);
    CREATE TABLE IF NOT EXISTS events(
        -- knock, enter, unknown, vehicle
        id INTEGER PRIMARY KEY, ts TEXT, camera TEXT, kind TEXT,
        identity TEXT, category TEXT, score REAL, snapshot TEXT);
    """)
    return con

def add_person(con, name, category):
    con.execute("INSERT OR IGNORE INTO people(name, category) VALUES(?,?)", (name, category))
    con.commit()
    return con.execute("SELECT id FROM people WHERE name=?", (name,)).fetchone()[0]

def add_embedding(con, person_id, emb, source):
    con.execute("INSERT INTO face_embeddings(person_id, embedding, source) VALUES(?,?,?)",
                (person_id, emb.astype(np.float32).tobytes(), source))
    con.commit()

def log_event(con, camera, kind, identity, category, score, snapshot):
    con.execute("INSERT INTO events(ts,camera,kind,identity,category,score,snapshot) "
```

```
            "VALUES(?,?,?,?,?,?,?)",
            (datetime.datetime.now().isoformat(timespec="seconds"), camera, kind,
             identity, category, float(score), snapshot))
    con.commit()
```

8.2 Collect the enrollment photos (the most important step for accuracy)

Most of your final accuracy comes from the quality of the enrollment data, not from the model. For every person:

- Collect **10 to 20 photos**: straight, slightly left/right, slightly up/down, with and without glasses or cap if they use them.
- Take some photos **with the door cameras themselves** (same angle, same lighting, same distance). This is the biggest accuracy boost. Save frames from the recordings or add a small script that saves face crops when you press a key.
- Include morning, evening and night (infrared) lighting.
- One face per photo, at least 200x200 pixels for the face.

```
dataset/
  owner__Ali/        1.jpg 2.jpg ... 15.jpg
  family__Mona/      1.jpg ...
  family__Cousin_Omar/
  dustman__Hassan/
  neighbor__Mr_Salem/
  neighbor__Mrs_Huda/
```

The folder name is category__Name. The enrollment script reads it, so you do not edit code to add a new person.

8.3 The enrollment script (src/enroll.py)

```
import os, cv2, sys
from faces import FaceEngine
import database as db

engine = FaceEngine(det_size=(640, 640))     # bigger det size for still photos
con = db.connect()
root = "dataset"
for folder in sorted(os.listdir(root)):
    if "__" not in folder:
        continue
    category, name = folder.split("__", 1)
    pid = db.add_person(con, name.replace("_", " "), category)
    added = 0
    for fn in sorted(os.listdir(os.path.join(root, folder))):
        img = cv2.imread(os.path.join(root, folder, fn))
        if img is None:
            continue
        faces = engine.analyse(img)
        if len(faces) != 1:                    # skip photos with 0 or several faces
            print(f"skip {folder}/{fn}: {len(faces)} faces")
            continue
        db.add_embedding(con, pid, faces[0]["embedding"], fn)
        added += 1
    print(f"{name}: {added} embeddings")
```

Note: This photo enrollment is your baseline. Part 2 (chapters T1 to T10, right after this chapter) builds a much larger dataset from your old footage and trains models that must beat this baseline.

What you do now: Create 3 folders with real photos (yourself, one family member, one friend) and run enroll.py. Every person must show at least 8 embeddings. Fix people who have few by adding better photos.

PART 2 - TRAINING PHASE

Chapters T1 to T10 turn your old camera footage into a custom dataset and train models on it. Do them after chapter 8 (you need the face engine and the database first). The photo enrollment in chapter 8 stays as your **baseline**: the training chapters must beat it on a fair test, and the numbers you measure become the core of your CV story.

T1. Training strategy and data plan

T1.1 What you will train

ID	Model	Input to output	Data needed	Difficulty
A	Classifier on frozen face embeddings (logistic regression)	512 numbers to person name	100+ face crops per person	Easy
B	ArcFace-loss adapter on top of the frozen face network	512 numbers to a better 512 numbers	200+ crops per person plus strangers	Medium
C	Fine-tune the last layers of the face network (optional)	160x160 face image to embedding	500+ crops per person, GPU (Colab)	Hard
D	Vehicle classifier (MobileNetV3, transfer learning)	Vehicle crop to which family car	100+ crops per car plus 'other'	Easy-Medium

T1.2 An honest note about what training can and cannot do

A good face network (ArcFace) was trained on millions of faces. You cannot train one from scratch with ten people. If you fine-tune *all* its layers on a few people, it often becomes better at those people but **worse at rejecting strangers** (it forgets what makes faces different). Professional teams therefore go step by step: baseline, then a cheap model on frozen embeddings (A), then a small trained adapter (B), and only then a partial fine-tune (C). You measure every step on the same test set and keep the winner. If C does not beat B, say so in your README: honest negative results impress interviewers more than inflated claims.

T1.3 How much data you need

Class	Minimum	Good
Each owner / family member	150 face crops from 6+ different videos	500+ crops from 15+ videos on different days and hours
Dustman, each neighbour	80 crops from 4+ videos	200+ crops from 8+ videos
Strangers (the 'unknown' class)	500 crops from many different people	2000+ crops (visitors, delivery people, passers-by, public datasets)
Each family car	80 crops from 4+ videos	300+ crops, day and night
Other vehicles	200 crops	1000+ crops

If someone appears in fewer than 4 videos, you cannot split their data fairly. Record more clips of them at the door, or keep them in the photo-enrollment baseline only.

T1.4 Five golden rules (read twice)

1. **Split by video, never by frame.** Frames 0.3 seconds apart are almost identical. If frame 100 is in training and frame 101 in test, your test score is fake. Whole videos (or whole days) go to train, validation or test.
2. **Use the same model pack for dataset creation and live use.** Embeddings from buffalo_sc and buffalo_l are not comparable. Pick one now (this guide uses buffalo_sc).
3. **Touch the test set once.** Tune on validation. Report on test at the very end.
4. **Label quality beats quantity.** 5% wrong labels can cost you several accuracy points and make threshold tuning meaningless.
5. **Consent and storage.** Your dataset is a collection of faces. Keep it on an encrypted disk, never upload it to GitHub, and tell the people in it.

T1.5 Folder layout for the whole training phase

```
door_ai/
  old_footage/            # your old recordings (read-only copies)
  data/
    videos.csv            # T2: list of videos with duration, fps, resolution
    segments.csv          # T2: time ranges where a person is present
    faces/                # T3: aligned 112x112 face crops
    faces_meta.csv        # T3: info per crop (video, time, size, sharpness)
    emb_crops.npy         # T4: embedding for each crop
    clusters/             # T4: automatic groups to label quickly
    labelled/             # T4: final labelled folders (category__Name, stranger__cNN)
    splits.csv            # T5: file, label, video, split (train/val/test)
    features.npz          # T6: embeddings (with augmentation) ready for training
    vehicles/             # T9: vehicle crops and splits
  models/                 # trained models (versioned: adapter_v1.pt ...)
  src/train/              # all scripts of this part
  eval/                   # metrics, confusion matrices, experiment log
```

What you do now: Copy your old footage into old_footage/ (keep the originals untouched elsewhere). Write a file data/classes.txt listing every person with category, for example: owner__Ali, family__Mona, family__Cousin_Omar, dustman__Hassan, neighbor__Mr_Salem. Create the folders above.

T2. Organise the old footage and find the useful parts

Security footage is 95% empty doorway. Before extracting faces, find the moments when a person is actually present. This saves many hours of processing.

T2.1 Make sure every video is readable

If your old footage comes from a DVR in a strange format (.dav, .avi, .264), convert it once to normal MP4:

```
ffmpeg -i input.dav -c:v libx264 -preset veryfast -crf 23 -an output.mp4
# convert a whole folder:
for f in old_footage_raw/*.dav; do ffmpeg -i "$f" -c:v libx264 -preset veryfast -crf 23 -an \
    "old_footage/$(basename "${f%.*}").mp4"; done
```

T2.2 Inventory script (src/train/inventory.py)

```
import cv2, csv, glob, os

rows = []
for path in sorted(glob.glob("old_footage/**/*.mp4", recursive=True)):
    cap = cv2.VideoCapture(path)
    fps = cap.get(cv2.CAP_PROP_FPS) or 25
    frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
    w, h = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)), int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    rows.append([path, round(frames / fps / 60, 1), round(fps, 1), f"{w}x{h}",
```

```
            os.path.getsize(path) // 1_000_000])
    cap.release()

with open("data/videos.csv", "w", newline="") as f:
    wr = csv.writer(f)
    wr.writerow(["video", "minutes", "fps", "resolution", "MB"])
    wr.writerows(rows)
print(len(rows), "videos,", round(sum(r[1] for r in rows) / 60, 1), "hours")
```

T2.3 The 'who appears' log (10 minutes of work that saves hours later)

Open data/videos.csv in a spreadsheet and add a column **who**. For the videos you remember (for example 'the day the dustman came', 'my cousin visited'), write the names. You do not need to be complete. It will help you recognise and label the clusters in T4, and it helps you balance the dataset: you will see which people have too few videos.

T2.4 Find the segments where a person is present (src/train/find_segments.py)

```
import cv2, csv, glob
from motion import MotionDetector
from detector import ObjectDetector

det = ObjectDetector(size=416)
out = csv.writer(open("data/segments.csv", "w", newline=""))
out.writerow(["video", "start_s", "end_s"])

for path in sorted(glob.glob("old_footage/**/*.mp4", recursive=True)):
    cap = cv2.VideoCapture(path)
    fps = cap.get(cv2.CAP_PROP_FPS) or 25
    step = max(1, int(fps / 2))                # analyse 2 frames per second
    md = MotionDetector()
    i, start, last = 0, None, None
    while cap.grab():                          # grab() skips decoding colour conversion: faster
        if i % step == 0:
            ok, frame = cap.retrieve()
            t = i / fps
            has_person = False
            if ok and md.detect(frame):
                has_person = len(det.persons(det.detect(frame))) > 0
            if has_person:
                start = t if start is None else start
                last = t
            # 5 s without a person closes the segment
            elif start is not None and t - last > 5:
                out.writerow([path, max(0, start - 2), last + 2])
                start = None
        i += 1
    if start is not None:
        out.writerow([path, max(0, start - 2), last + 2])
    cap.release()
    print("done", path)
```

Note: Speed: decoding is the slow part. On an old laptop expect roughly 3 to 8 times real time, so 24 hours of footage can take 3 to 8 hours. Run it overnight, process the most useful days first, and use the lower-resolution recordings if you have them. The script is safe to stop; process folders separately by changing the glob pattern.

What you do now: Run inventory.py and find_segments.py. Open data/segments.csv and check 5 random segments in VLC (jump to start_s): a person must really be there. Write down total minutes of segments; you need several hours in total to get enough crops.

T3. Extract aligned face crops from the segments

Now we turn video segments into a pile of face images. Each crop is **aligned** (rotated and scaled so the eyes and mouth are in standard positions) to 112x112 pixels, which is exactly what the recognition network expects. We also record where every crop came from (video and time), because we need it for splitting.

```
# src/train/extract_faces.py
import cv2, csv, os, numpy as np
from insightface.app import FaceAnalysis
from insightface.utils import face_align

MODEL = "buffalo_sc"        # MUST be the same pack as the live pipeline (see rule 2 in T1)
SAMPLE_FPS = 3              # 3 crops per second per person is plenty
MIN_SIZE, MIN_DET, MIN_SHARP = 50, 0.6, 30

app = FaceAnalysis(name=MODEL, providers=["CPUExecutionProvider"])
app.prepare(ctx_id=-1, det_size=(640, 640))

os.makedirs("data/faces", exist_ok=True)
segments = list(csv.DictReader(open("data/segments.csv")))
meta = csv.writer(open("data/faces_meta.csv", "w", newline=""))
meta.writerow(["file", "video", "t_s", "det", "size", "sharp"])
n = 0

for seg in segments:
    cap = cv2.VideoCapture(seg["video"])
    vid = os.path.splitext(seg["video"])[0].replace("/", "_").replace(" ", "_")    # unique video id
    t, end = float(seg["start_s"]), float(seg["end_s"])
    while t < end:
        cap.set(cv2.CAP_PROP_POS_MSEC, t * 1000)
        ok, frame = cap.read()
        t += 1.0 / SAMPLE_FPS
        if not ok:
            continue
        for f in app.get(frame):
            x1, y1, x2, y2 = f.bbox.astype(int)
            size = min(x2 - x1, y2 - y1)
            if size < MIN_SIZE or f.det_score < MIN_DET:
                continue
            aligned = face_align.norm_crop(frame, f.kps, image_size=112)
            sharp = cv2.Laplacian(cv2.cvtColor(aligned, cv2.COLOR_BGR2GRAY), cv2.CV_64F).var()
            if sharp < MIN_SHARP:
                continue
            fn = f"data/faces/{vid}__{int(t * 1000):08d}__{n}.jpg"
            cv2.imwrite(fn, aligned)
            meta.writerow([fn, vid, round(t, 2), round(float(f.det_score), 3), size, round(sharp,
                    1)])
            n += 1
    cap.release()
    print(seg["video"], "total crops so far:", n)
```

Important details

- Several faces can appear in one frame (a visitor with the family). That is good: every face becomes a crop, and labelling decides who is who.
- The file name carries the video id. Never rename crops; later steps recover the video from the file name.
- If you get very few crops, lower MIN_SIZE to 40 or MIN_SHARP to 20, or process the main-stream recordings instead of the sub-stream ones.
- Seeking with CAP_PROP_POS_MSEC is not frame-exact for some codecs, but it is accurate enough here.

What you do now: Run extract_faces.py. Open data/faces in a file manager with large thumbnails and look at 50 random crops. They should be upright, centred and mostly sharp. If many are rotated or not faces, raise MIN_DET to 0.7. Record how many crops you have.

T4. Label the dataset the fast way (embeddings + clustering)

Labelling thousands of crops one by one is painful and error-prone. Instead we let the computer group similar faces automatically, and you only name each *group*. This is a standard technique (semi-automatic labelling) and a good talking point.

T4.1 Compute an embedding for every crop (src/train/embed.py)

```
# src/train/embed.py
import os, cv2, numpy as np
from insightface.model_zoo import get_model

# buffalo_sc recognition model. If you use buffalo_l the file is w600k_r50.onnx
REC = os.path.expanduser("~/.insightface/models/buffalo_sc/w600k_mbf.onnx")
rec = get_model(REC, providers=["CPUExecutionProvider"])
rec.prepare(ctx_id=-1)

def embed_batch(imgs):                      # list of 112x112 BGR images
    f = rec.get_feat(imgs)                  # shape (N, 512)
    return (f / np.linalg.norm(f, axis=1, keepdims=True)).astype(np.float32)

def embed_files(files, batch=64):
    out = []
    for i in range(0, len(files), batch):
        imgs = [cv2.imread(f) for f in files[i:i + batch]]
        out.append(embed_batch(imgs))
    return np.concatenate(out)

if __name__ == "__main__":
    import csv
    files = [r["file"] for r in csv.DictReader(open("data/faces_meta.csv"))]
    np.save("data/emb_crops.npy", embed_files(files))
    print("saved", len(files), "embeddings")
```

T4.2 Cluster and create contact sheets (src/train/cluster_faces.py)

```
import csv, os, shutil, cv2, numpy as np
from sklearn.cluster import DBSCAN

E = np.load("data/emb_crops.npy")
meta = list(csv.DictReader(open("data/faces_meta.csv")))
EPS = 0.45        # cosine distance. Lower = purer but more, smaller clusters. Tune it.
labels = DBSCAN(eps=EPS, min_samples=6, metric="cosine").fit_predict(E)

def montage(files, cols=10, size=80, maxn=60):
    tiles = [cv2.resize(cv2.imread(f), (size, size)) for f in files[:maxn]]
    while len(tiles) % cols:
        tiles.append(np.zeros((size, size, 3), np.uint8))
    return np.vstack([np.hstack(tiles[i:i + cols]) for i in range(0, len(tiles), cols)])

shutil.rmtree("data/clusters", ignore_errors=True)
print("cluster  faces  videos")
for c in sorted(set(labels), key=lambda c: -(labels == c).sum()):
    name = "noise" if c == -1 else f"cluster_{c:03d}"
    idx = np.where(labels == c)[0]
    files = [meta[i]["file"] for i in idx]
    os.makedirs(f"data/clusters/{name}", exist_ok=True)
    for f in files:
        shutil.copy(f, f"data/clusters/{name}/{os.path.basename(f)}")
    cv2.imwrite(f"data/clusters/{name}_sheet.jpg", montage(files))
    print(f"{name:12s} {len(files):5d} {len(set(meta[i]['video'] for i in idx)):5d}")
```

T4.3 Name the clusters (this is the manual part, about 1 to 2 hours)

1. Open the contact sheets `data/clusters/cluster_XXX_sheet.jpg` one by one. Each shows up to 60 faces of one cluster.
2. If all faces are one person you know, move the cluster into a labelled folder named **category__Name**. Several clusters of the same person all go to the same folder.
3. If a cluster has two different people mixed, open the folder with big thumbnails and delete or move the wrong faces to the correct place.
4. Clusters of people you do not know (delivery people, visitors, passers-by) go to **stranger__cNN**, one folder per cluster. Do not merge them; each stranger group is a useful negative class later.
5. Skim the **noise** folder: good clear faces of known people can be moved to their person folder, the rest can be dropped or kept as strangers.
6. Use your 'who appears' log: a cluster found only in the video where the dustman came is almost certainly the dustman.

```
mkdir -p data/labelled/owner__Ali data/labelled/dustman__Hassan data/labelled/stranger__c01
cp data/clusters/cluster_000/*.jpg data/labelled/owner__Ali/
# second cluster of the same person
cp data/clusters/cluster_004/*.jpg data/labelled/owner__Ali/
cp data/clusters/cluster_007/*.jpg data/labelled/dustman__Hassan/
cp data/clusters/cluster_012/*.jpg data/labelled/stranger__c01/
```

T4.4 Quality check before you continue (do not skip)

- Open every labelled folder, sort by name, and scan 100 random crops: no wrong person allowed. Fix mistakes now.
- Run a self-consistency test: for each labelled image compute the cosine similarity to the average embedding of its folder. Images with a very low value (for example below 0.35) are likely mislabelled; review them.
- Count crops and distinct videos per folder and compare with the targets in T1.3.
- People who are mostly seen from far away or in the dark will have worse data: that is real and should show up in your final per-person results.

```
# src/train/label_check.py
import glob, os, numpy as np
from embed import embed_files

for folder in sorted(glob.glob("data/labelled/*")):
    files = glob.glob(folder + "/*.jpg")
    if len(files) < 5:
        continue
    E = embed_files(files)
    center = E.mean(0)
    center /= np.linalg.norm(center)
    sims = E @ center
    bad = [os.path.basename(files[i]) for i in np.where(sims < 0.35)[0]]
    name = os.path.basename(folder)
    print(f"{name:28s} n={len(files):4d}  mean_sim={sims.mean():.2f}  suspicious={len(bad)}")
    for b in bad[:10]:
        print("     check:", b)
```

Note: This check is for known people. For stranger folders low similarity is normal if you merged several strangers; keep one cluster per stranger folder.

What you do now: Label at least: yourself, 2 family members, the dustman, 1 neighbour, and 5+ stranger clusters. Run label_check.py and fix the suspicious images. Write the final count of crops and videos per folder in a small table; you will paste it in the data card (T5).

T5. Final dataset, splits and data card

T5.1 Split by video (src/train/make_splits.py)

Each person's videos are shuffled and divided: about 65% train, 15% validation, 20% test. A video lives in exactly one split. If a person has only 3 videos, one goes to each split; fewer than 3 means that person cannot be evaluated fairly yet.

```
import csv, glob, os, random
from collections import defaultdict

meta_rows = csv.DictReader(open("data/faces_meta.csv"))
video_of = {os.path.basename(r["file"]): r["video"] for r in meta_rows}
rows = []
for folder in sorted(glob.glob("data/labelled/*")):
    label = os.path.basename(folder)
    by_video = defaultdict(list)
    for f in glob.glob(folder + "/*.jpg"):
        by_video[video_of[os.path.basename(f)]].append(f)
    vids = sorted(by_video)
    random.Random(42).shuffle(vids)
    n = len(vids)
    if n < 3:
        print("WARNING: only", n, "videos for", label, "- all goes to train, cannot be evaluated")
        n_test = n_val = 0
    else:
        n_test = max(1, round(0.20 * n))
        n_val = max(1, round(0.15 * n))
    test_v, val_v = set(vids[:n_test]), set(vids[n_test:n_test + n_val])
    for v in vids:
        split = "test" if v in test_v else "val" if v in val_v else "train"
        for f in by_video[v]:
            rows.append([f, label, v, split])

with open("data/splits.csv", "w", newline="") as fh:
    w = csv.writer(fh)
    w.writerow(["file", "label", "video", "split"])
    w.writerows(rows)

# summary table
count = defaultdict(lambda: defaultdict(int))
for _, label, _, split in rows:
    count[label][split] += 1
print(f"{'label':30s} train   val  test")
for label in sorted(count):
    c = count[label]
    print(f"{label:30s} {c['train']:5d} {c['val']:5d} {c['test']:5d}")
```

T5.2 Write the data card (a one-page document in your repo)

Create docs/DATA_CARD.md. It shows professionalism and is usually missing in student projects. Include:

- Source: your own door cameras, dates covered, resolution, day/night proportion.
- Classes and counts per split (paste the table printed above), and how strangers were collected.
- Labelling method: automatic clustering plus manual review, and the quality checks you performed.
- Splitting method: by video, with random seed 42.
- Known limitations: few people, only two camera angles, little data of people wearing masks or hats, seasonal clothing changes.
- Privacy: consent obtained, storage location, retention period, who can access it.

What you do now: Run make_splits.py. Every person should have crops in train, validation and test. If someone shows 0 in val or test, add more videos for them or accept that they are evaluated only by photo enrollment.

T6. Training A: a classifier on frozen embeddings

The recognition network stays frozen. We train a small, fast model that maps a 512-number embedding to a person name. To make it robust we train on **augmented** versions of the crops: brighter, darker, mirrored, blurred, low-resolution (a far-away face) and grayscale (infrared night look). This teaches the system what the real door conditions look like.

T6.1 Augmentation and feature building (src/train/build_features.py)

```
import csv, numpy as np, cv2
from embed import embed_batch

rng = np.random.default_rng(0)

def augment(img):
    out = img.copy()
    if rng.random() < 0.5:
        out = cv2.flip(out, 1)
    a, b = rng.uniform(0.6, 1.4), rng.uniform(-30, 30)                # contrast and brightness
    out = np.clip(out.astype(np.float32) * a + b, 0, 255).astype(np.uint8)
    # far-away / low-resolution face
    if rng.random() < 0.4:
        s = int(rng.integers(40, 90))
        out = cv2.resize(cv2.resize(out, (s, s)), (112, 112))
    if rng.random() < 0.3:
        k = int(rng.choice([3, 5]))
        out = cv2.GaussianBlur(out, (k, k), 0)
    if rng.random() < 0.3:                                            # infrared / night look
        out = cv2.cvtColor(cv2.cvtColor(out, cv2.COLOR_BGR2GRAY), cv2.COLOR_GRAY2BGR)
    if rng.random() < 0.3:
        out = np.clip(out.astype(np.float32) + rng.normal(0, 8, out.shape), 0, 255).astype(np.uint8)
    return out

rows = list(csv.DictReader(open("data/splits.csv")))
names = sorted(set(r["label"] for r in rows))
idx = {n: i for i, n in enumerate(names)}
N_AUG = 3                                          # augmented copies per training image

def build(split, aug):
    X, y = [], []
    sel = [r for r in rows if r["split"] == split]
    for i in range(0, len(sel), 64):
        batch = sel[i:i + 64]
        imgs = [cv2.imread(r["file"]) for r in batch]
        X.append(embed_batch(imgs))
        y += [idx[r["label"]] for r in batch]
        for _ in range(aug):
            X.append(embed_batch([augment(im) for im in imgs]))
            y += [idx[r["label"]] for r in batch]
    return np.concatenate(X), np.array(y)

Xtr, ytr = build("train", N_AUG)
Xva, yva = build("val", 0)
Xte, yte = build("test", 0)
np.savez("data/features.npz", Xtr=Xtr, ytr=ytr, Xva=Xva, yva=yva, Xte=Xte, yte=yte,
        names=np.array(names))
print("train", Xtr.shape, "val", Xva.shape, "test", Xte.shape)
```

Note: Only the training images are augmented. Validation and test stay untouched so the scores reflect real data.

T6.2 The open-set metrics (src/train/evalutil.py)

A door system is *open-set*: most visitors are not in your database. So accuracy alone is not enough. We measure three things at a given confidence threshold t:

- **FRR**: known people that were not correctly accepted (missed or confused).
- **FAR (stranger)**: strangers that were accepted as a known person. This is the dangerous error.
- **misID**: known people accepted but as the wrong person.

```
import numpy as np

def open_set_metrics(pred, conf, y, known_idx, t):
    known_idx = np.array(known_idx)
    is_known = np.isin(y, known_idx)                          # true identity is a known person
    accepted = (conf >= t) & np.isin(pred, known_idx)         # system says "this is a known person"
    correct = accepted & (pred == y)
    frr = 1 - correct[is_known].mean()
    far = accepted[~is_known].mean() if (~is_known).any() else 0.0
    misid = (accepted & (pred != y))[is_known].mean()
    return {"t": round(float(t), 3), "FRR": round(float(frr), 4),
            "FAR_stranger": round(float(far), 4), "misID_known": round(float(misid), 4)}

def pick_threshold(pred, conf, y, known_idx, max_far=0.01):
    best = None
    for t in np.linspace(0.05, 0.99, 95):
        m = open_set_metrics(pred, conf, y, known_idx, t)
        if m["FAR_stranger"] <= max_far and (best is None or m["FRR"] < best["FRR"]):
            best = m
    return best
```

T6.3 Train and evaluate (src/train/train_classifier.py)

```
import numpy as np, joblib
from sklearn.linear_model import LogisticRegression
from evalutil import open_set_metrics, pick_threshold

d = np.load("data/features.npz", allow_pickle=True)
names = list(d["names"])
known_idx = [i for i, n in enumerate(names) if not n.startswith("stranger__")]

clf = LogisticRegression(C=10, max_iter=3000, class_weight="balanced")
clf.fit(d["Xtr"], d["ytr"])

def run(X, y, t=None):
    P = clf.predict_proba(X)
    pred, conf = clf.classes_[P.argmax(1)], P.max(1)
    return pred, conf

pred, conf = run(d["Xva"], d["yva"])
best = pick_threshold(pred, conf, d["yva"], known_idx, max_far=0.01)
print("threshold chosen on VALIDATION:", best)

joblib.dump({"clf": clf, "names": names, "threshold": best["t"]}, "models/classifier_v1.joblib")
# Do NOT look at the test set yet. It is used once in T10 for the final comparison.
```

Things to try (one at a time, write results in your experiment log)

- C (regularisation): 1, 10, 100. Smaller C = simpler model.
- N_AUG: 0, 1, 3, 6. Does augmentation help on validation?
- Remove the stranger classes to see how much they help reject unknown people (they should help a lot).
- Replace LogisticRegression by a linear SVM or a small MLP and compare.

What you do now: Run build_features.py, then train_classifier.py. Write down the validation numbers: FRR, FAR_stranger and misID at the chosen threshold. This is model A.

T7. Training B: an ArcFace adapter in PyTorch

Now we train a real neural-network component. The idea: keep the big face network frozen, and learn a small **adapter** that gently reshapes embeddings so that *your* people are better separated from each other and from strangers, under *your* camera conditions. We train it with **ArcFace loss**, the same loss family used to train state-of-the-art face recognition, which pushes classes apart by an angular margin.

T7.1 The two modules (src/adapter.py, used for training and at runtime)

```
import math, torch, torch.nn as nn, torch.nn.functional as F

class Adapter(nn.Module):
    # residual adapter, initialised as identity, so it starts exactly as good as the frozen baseline
    def __init__(self, d=512, p=0.1):
        super().__init__()
        self.drop = nn.Dropout(p)
        self.lin = nn.Linear(d, d)
        nn.init.zeros_(self.lin.weight)
        nn.init.zeros_(self.lin.bias)

    def forward(self, x):
        return F.normalize(x + self.lin(self.drop(x)), dim=1)

class ArcMargin(nn.Module):
    # ArcFace head: cos(theta + m) for the true class, scaled by s
    def __init__(self, d, n_classes, s=30.0, m=0.30):
        super().__init__()
        self.W = nn.Parameter(torch.randn(n_classes, d) * 0.01)
        self.s, self.m = s, m

    def forward(self, z, y=None):
        cos = F.linear(F.normalize(z), F.normalize(self.W)).clamp(-1 + 1e-7, 1 - 1e-7)
        if y is None:
            return cos * self.s
        target = torch.cos(torch.acos(cos) + self.m)
        onehot = F.one_hot(y, cos.size(1)).bool()
        return torch.where(onehot, target, cos) * self.s
```

T7.2 The training script (src/train/train_adapter.py)

```
import sys, numpy as np, torch, torch.nn.functional as F
sys.path.append("src")
from adapter import Adapter, ArcMargin
from evalutil import open_set_metrics, pick_threshold

torch.manual_seed(0)
d = np.load("data/features.npz", allow_pickle=True)
names = list(d["names"])
known_idx = [i for i, n in enumerate(names) if not n.startswith("stranger__")]
Xtr, ytr = torch.tensor(d["Xtr"]), torch.tensor(d["ytr"], dtype=torch.long)
Xva, yva = torch.tensor(d["Xva"]), torch.tensor(d["yva"], dtype=torch.long)

adapter, head = Adapter(), ArcMargin(512, len(names))
opt = torch.optim.AdamW(list(adapter.parameters()) + list(head.parameters()), lr=1e-3,
        weight_decay=1e-2)

def prototypes(Z, y):
    return F.normalize(torch.stack([Z[y == c].mean(0) for c in range(len(names))]), dim=1)

best_score, patience, best_state = -1, 0, None
for epoch in range(200):
    adapter.train(); head.train()
    perm = torch.randperm(len(Xtr))
    total = 0.0
    for i in range(0, len(perm), 128):
        idx = perm[i:i + 128]
        loss = F.cross_entropy(head(adapter(Xtr[idx]), ytr[idx]), ytr[idx])
        opt.zero_grad(); loss.backward(); opt.step()
        total += loss.item() * len(idx)

    adapter.eval()
```

```
    with torch.no_grad():
        P = prototypes(adapter(Xtr), ytr)
        sims = adapter(Xva) @ P.T
        conf, pred = sims.max(1)
    pred, conf, y = pred.numpy(), conf.numpy(), yva.numpy()
    m = pick_threshold(pred, conf, y, known_idx, max_far=0.01)
    score = 1 - m["FRR"] if m else 0
    frr = m["FRR"] if m else "n/a"
    print(f"epoch {epoch:3d} loss {total / len(Xtr):.3f}  val FRR@FAR<=1%: {frr}")
    if score > best_score:
        best_score, patience = score, 0
        best_state = ({k: v.clone() for k, v in adapter.state_dict().items()}, P.clone(), m)
    else:
        patience += 1
        if patience >= 15:                      # early stopping
            break

state, P, m = best_state
cats = [n.split("__", 1)[0] for n in names]
torch.save({"adapter": state, "protos": P, "names": names, "cats": cats, "threshold": m["t"]},
           "models/adapter_v1.pt")
print("saved. validation result:", m)
```

Why this design is sound

- The adapter starts as the identity, so training cannot begin worse than the baseline; early stopping protects against overfitting.
- Each stranger cluster is its own class: the model learns that there are many different 'other people', which improves rejection of new strangers.
- At runtime we compare to class **prototypes** (average embedding per class). Adding a new person later needs no retraining of the network, only a new prototype.
- The threshold is chosen on validation with a hard FAR limit of 1%.

Experiments for your report

- Margin m: 0.0 (plain softmax), 0.2, 0.3, 0.5. Scale s: 20, 30, 64.
- Dropout p: 0, 0.1, 0.3. Weight decay: 0, 1e-2, 1e-1.
- With and without stranger classes; with and without augmented samples.
- Plot training loss and validation FRR per epoch (matplotlib) and save the figure to eval/.

Note: If the validation score never improves beyond the first epoch, your dataset is too small or too clean: the baseline is already near its ceiling. That is a valid finding. Collect more varied data (night, far, side views) before changing the model.

What you do now: Run train_adapter.py. You should see the validation FRR (at FAR 1%) go down over some epochs and then stop improving. Write the final validation numbers next to model A. Keep both models: they are compared on the test set in T10.

T8. Training C (optional, advanced): fine-tune the face network

This is the step most students attempt first and should attempt last. It fine-tunes real layers of a face network on your crops. It needs a GPU to be practical, so use **Google Colab** (free tier GPU): upload data/labelled and data/splits.csv (as a zip) and run the script there. It changes the embedding model itself, so everything downstream must use that model (the dataset embeddings, the prototypes, the live pipeline).

T8.1 Setup

```
pip install facenet-pytorch
# model: InceptionResnetV1 pretrained on VGGFace2, 160x160 RGB input, 512-d embedding
```

T8.2 Fine-tuning code (outline you complete and test)

```
import csv, cv2, numpy as np, torch, torch.nn.functional as F
from facenet_pytorch import InceptionResnetV1
from adapter import ArcMargin
# move augment() from T6 into src/train/augment.py and import it in both scripts
from augment import augment

dev = "cuda" if torch.cuda.is_available() else "cpu"
rows = list(csv.DictReader(open("data/splits.csv")))
names = sorted(set(r["label"] for r in rows)); idx = {n: i for i, n in enumerate(names)}

class FaceDS(torch.utils.data.Dataset):
    def __init__(self, split, train):
        self.r = [x for x in rows if x["split"] == split]
        self.train = train
    def __len__(self):
        return len(self.r)
    def __getitem__(self, i):
        img = cv2.imread(self.r[i]["file"])
        if self.train:
            img = augment(img)
        img = cv2.cvtColor(cv2.resize(img, (160, 160)), cv2.COLOR_BGR2RGB)
        x = (torch.tensor(img).permute(2, 0, 1).float() - 127.5) / 128.0
        return x, idx[self.r[i]["label"]]

model = InceptionResnetV1(pretrained="vggface2").to(dev)
for p in model.parameters():
    p.requires_grad = False
for module in (model.block8, model.last_linear, model.last_bn):    # unfreeze only the last parts
    for p in module.parameters():
        p.requires_grad = True

head = ArcMargin(512, len(names)).to(dev)
opt = torch.optim.AdamW([{"params": [p for p in model.parameters() if p.requires_grad], "lr": 1e-4},
                         {"params": head.parameters(), "lr": 1e-3}], weight_decay=1e-2)
loader = torch.utils.data.DataLoader(FaceDS("train", True), batch_size=64, shuffle=True)

for epoch in range(10):
    # keep BatchNorm statistics of frozen layers fixed
    model.eval()
    model.block8.train(); model.last_linear.train(); model.last_bn.train()
    for x, y in loader:
        x, y = x.to(dev), y.to(dev)
        loss = F.cross_entropy(head(model(x), y), y)
        opt.zero_grad(); loss.backward(); opt.step()
    print("epoch", epoch, "loss", float(loss))
    # TODO: compute validation FRR at FAR <= 1% exactly as in T7 and keep the best epoch

torch.save({"model": model.state_dict(), "names": names}, "models/facenet_ft_v1.pt")
```

Rules for doing C responsibly

- Very small learning rate (1e-4 or lower) and few epochs. Watch the validation FAR for strangers: if it grows while accuracy on known people grows, you are overfitting.

- Compare against model B on the **same test split**. Keep C only if it wins by a clear margin.
- You must also use the same 160x160 preprocessing at runtime (crop the face with the InsightFace box or landmarks, resize to 160, normalise as above).
- Report training time and hardware. A model that is 0.3% better but needs a GPU is not an improvement for an old laptop.

What you do now: Only start T8 after T6, T7 and T10 evaluation are done. Document it as an experiment, even if it fails.

T9. Train the family-car recogniser

The same workflow applies to vehicles: extract crops from old footage, cluster, label, split by video, train. You train a small image classifier with **transfer learning** (start from a network pretrained on ImageNet).

T9.1 Extract vehicle crops (src/train/extract_vehicles.py)

```
import cv2, csv, glob, os
from detector import ObjectDetector

det = ObjectDetector(size=640, conf=0.4)
os.makedirs("data/vehicles/raw", exist_ok=True)
meta = csv.writer(open("data/vehicles_meta.csv", "w", newline=""))
meta.writerow(["file", "video", "t_s"])
n = 0
for path in sorted(glob.glob("old_footage/**/*.mp4", recursive=True)):
    vid = os.path.splitext(path)[0].replace("/", "_").replace(" ", "_")
    cap = cv2.VideoCapture(path)
    fps = cap.get(cv2.CAP_PROP_FPS) or 25
    step = int(fps)                                  # 1 frame per second
    i = 0
    while cap.grab():
        if i % step == 0:
            ok, frame = cap.retrieve()
            if ok:
                for d in det.vehicles(det.detect(frame)):
                    x1, y1, x2, y2 = d["box"]
                    if (x2 - x1) < 120:              # ignore tiny, far vehicles
                        continue
                    fn = f"data/vehicles/raw/{vid}__{i}__{n}.jpg"
                    cv2.imwrite(fn, frame[max(0, y1):y2, max(0, x1):x2])
                    meta.writerow([fn, vid, round(i / fps, 1)])
                    n += 1
        i += 1
    cap.release()
```

T9.2 Label and split

- Cluster the crops with image features (MobileNet embeddings plus DBSCAN, same code pattern as T4) or simply sort the thumbnails in a file manager and drag them into folders by car.
- Folders: **car__Dad**, **car__Uncle**, ... and **other** (every vehicle that is not yours).
- Split by video exactly like T5 (copy make_splits.py and change the paths). Result: data/vehicles/train/<class>/, val/ and test/ folders, which torchvision can read directly.

T9.3 Train (src/train/train_vehicle.py)

```
import torch, torch.nn as nn
from torchvision import datasets, transforms, models

norm = transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
tf_train = transforms.Compose([transforms.Resize((160, 160)), transforms.ColorJitter(0.4, 0.4, 0.3),
                               transforms.RandomHorizontalFlip(), transforms.RandomRotation(5),
                               transforms.ToTensor(), norm])
tf_eval = transforms.Compose([transforms.Resize((160, 160)), transforms.ToTensor(), norm])

train = datasets.ImageFolder("data/vehicles/train", tf_train)
val = datasets.ImageFolder("data/vehicles/val", tf_eval)
tl = torch.utils.data.DataLoader(train, batch_size=32, shuffle=True)
vl = torch.utils.data.DataLoader(val, batch_size=64)

m = models.mobilenet_v3_small(weights=models.MobileNet_V3_Small_Weights.DEFAULT)
for p in m.parameters():
    p.requires_grad = False
m.classifier[3] = nn.Linear(m.classifier[3].in_features, len(train.classes))   # new head

def evaluate():
    m.eval(); ok = n = 0
    with torch.no_grad():
        for x, y in vl:
            ok += (m(x).argmax(1) == y).sum().item(); n += len(y)
    return ok / n

def fit(params, lr, epochs):
    opt = torch.optim.AdamW(params, lr=lr, weight_decay=1e-2)
    for e in range(epochs):
        m.train()
        for x, y in tl:
            loss = nn.functional.cross_entropy(m(x), y, label_smoothing=0.1)
            opt.zero_grad(); loss.backward(); opt.step()
        print("epoch", e, "val acc", round(evaluate(), 3))

fit(m.classifier[3].parameters(), 1e-3, 6)              # phase 1: only the new head
for p in m.features[-3:].parameters():
    p.requires_grad = True                               # phase 2: unfreeze the last blocks
fit([p for p in m.parameters() if p.requires_grad], 1e-4, 6)
torch.save({"model": m.state_dict(), "classes": train.classes}, "models/vehicle_v1.pt")
```

- Use the classifier together with the plate reader (chapter 11): a car is accepted only when both agree, or when one is very confident.
- Open-set again: the 'other' class plus a softmax-confidence threshold (chosen on validation) decides unknown vehicles.
- Night images: add dark and infrared-like augmentation (reduce ColorJitter brightness range and add grayscale) if your night recall is low.

What you do now: Extract vehicle crops, label them, split by video, train. Report accuracy on validation and a per-class table; test only in T10.

T10. Final evaluation, experiment log, integration and retraining loop

T10.1 Compare all models on the untouched test split

Now (and only now) run the test split once for each model, using the thresholds you chose on validation. Use the same function for every model:

```
# src/train/final_eval.py
import sys, json, numpy as np, torch, joblib, torch.nn.functional as F
sys.path.append("src")
from adapter import Adapter
from evalutil import open_set_metrics
from sklearn.metrics import confusion_matrix

d = np.load("data/features.npz", allow_pickle=True)
names = list(d["names"])
known_idx = [i for i, n in enumerate(names) if not n.startswith("stranger__")]
Xtr, ytr, Xte, yte = d["Xtr"], d["ytr"], d["Xte"], d["yte"]
results = {}

# Baseline: prototype matching on raw embeddings (the chapter 9 method). Threshold from validation.
def protos(X, y):
    P = np.stack([X[y == c].mean(0) for c in range(len(names))])
    return P / np.linalg.norm(P, axis=1, keepdims=True)
from evalutil import pick_threshold
P0 = protos(Xtr, ytr)
sv = d["Xva"] @ P0.T
t0 = pick_threshold(sv.argmax(1), sv.max(1), d["yva"], known_idx)["t"]
st = Xte @ P0.T
results["baseline_prototypes"] = open_set_metrics(st.argmax(1), st.max(1), yte, known_idx, t0)

# Model A: logistic regression
a = joblib.load("models/classifier_v1.joblib")
P = a["clf"].predict_proba(Xte)
results["A_logreg"] = open_set_metrics(a["clf"].classes_[P.argmax(1)], P.max(1), yte, known_idx,
        a["threshold"])

# Model B: ArcFace adapter
ck = torch.load("models/adapter_v1.pt", map_location="cpu")
ad = Adapter(); ad.load_state_dict(ck["adapter"]); ad.eval()
with torch.no_grad():
    s = ad(torch.tensor(Xte)) @ ck["protos"].T
conf, pred = s.max(1)
results["B_adapter"] = open_set_metrics(pred.numpy(), conf.numpy(), yte, known_idx, ck["threshold"])

print(json.dumps(results, indent=2))
json.dump(results, open("eval/test_results.json", "w"), indent=2)

# confusion matrix of model B on known people (rows = truth, columns = prediction)
cm = confusion_matrix(yte, pred.numpy(), labels=range(len(names)))
np.savetxt("eval/confusion_B.csv", cm, fmt="%d", delimiter=",")
```

T10.2 Experiment log

Keep a table (eval/experiments.md or a CSV) with one row per experiment: date, what changed, data version, validation numbers, notes. Example columns:

Run	Change	Val FRR @ FAR 1%	Decision
A1	LogReg C=10, no augmentation	(your number)	Baseline for A
A2	Same, with 3 augmented copies	(your number)	Keep augmentation if better
B1	Adapter, m=0.3, s=30	(your number)	First adapter
B2	Adapter without stranger classes	(your number)	Shows value of strangers

Run	Change	Val FRR @ FAR 1%	Decision
B3	Adapter, m=0.5	(your number)	Margin study

Optional tools: MLflow or Weights & Biases (both free for personal use) track runs automatically. A simple CSV is enough and is easier to explain.

T10.3 Analyse the failures (this is where you learn the most)

- Save every wrongly accepted stranger and every missed known person from the test split into folders and look at them. Typical causes: infrared glare, hat or mask, profile view, motion blur, very small faces.
- Break down FRR by camera, by day/night and by face size. A table 'FRR by condition' is a very strong README element.
- Check the confusion matrix for pairs of family members who get mixed up (siblings): they need more data or a stricter threshold.
- Plot a histogram of cosine scores for 'known' versus 'stranger': the overlap shows how well-separated your classes are.

T10.4 Plug the trained model into the live pipeline

```
# add to src/matcher.py
import torch
from adapter import Adapter

class TrainedMatcher:
    def __init__(self, path="models/adapter_v1.pt"):
        ck = torch.load(path, map_location="cpu")
        self.adapter = Adapter()
        self.adapter.load_state_dict(ck["adapter"])
        self.adapter.eval()
        self.protos = ck["protos"]
        # e.g. "owner__Ali", "stranger__c03"
        self.full = ck["names"]
        self.cats = [n.split("__", 1)[0] for n in self.full]
        self.names = [n.split("__", 1)[1].replace("_", " ") for n in self.full]
        self.threshold = ck["threshold"]

    @torch.no_grad()
    def match(self, emb):
        z = self.adapter(torch.from_numpy(emb)[None])
        sims = (z @ self.protos.T)[0]
        i = int(sims.argmax())
        score = float(sims[i])
        if self.cats[i] == "stranger" or score < self.threshold:
            return "unknown", "unknown", score
        return self.names[i], self.cats[i], score

# in main.py replace:  matcher = FaceMatcher(con, threshold=...)
#            with:     matcher = TrainedMatcher("models/adapter_v1.pt")
```

Keep the old FaceMatcher too and add a config switch (matcher: baseline | trained). Being able to compare both on live footage is a nice demo.

T10.5 Close the loop: collect hard examples and retrain

Models get better when they learn from their own mistakes. Add this loop; it is called **active learning** and is exactly how industry improves models.

1. In the live pipeline, whenever a face is **unknown** or its score is within 0.05 of the threshold, save the aligned crop to `to_review/YYYY-MM-DD/` (plus camera and time in the file name).
2. Once a week, spend 15 minutes sorting these crops into labelled folders (correct person, stranger, or delete).

3. Copy them into data/labelled, rebuild the splits (new videos stay together), retrain A and B, and evaluate again.
4. Promote the new model only if it beats the current one on the **same** test split (never mix new hard examples into the old test split; create a new test version and keep the old one for comparison).
5. Save models with version numbers (adapter_v2.pt, v3 ...) and write a one-paragraph change note in docs/MODEL_CARD.md.

T10.6 Model card (docs/MODEL_CARD.md)

- Intended use and non-intended use (not for identification of the public, not for legal evidence).
- Training data summary (link to the data card), model versions, metrics on the test split and per-condition breakdown.
- Known failure modes and what you did about them. Fairness note: results for each person, not only the average.
- How to retrain and how to roll back.

What you do now: Run final_eval.py once, fill the comparison table (baseline, A, B, and C if you did it), write the data card and model card, and add the table and two failure-analysis figures to your README. Then switch main.py to TrainedMatcher and test live at the door with a friend as a stranger.

9. Matching, thresholds and the unknown class

The matcher loads all embeddings into one matrix and computes cosine similarity with a single matrix multiplication. For hundreds of embeddings this takes microseconds, so you do not need a vector database.

```
# src/matcher.py
import numpy as np

class FaceMatcher:
    def __init__(self, con, threshold=0.45):
        rows = con.execute("""SELECT p.name, p.category, e.embedding FROM face_embeddings e
                              JOIN people p ON p.id = e.person_id""").fetchall()
        self.names = [r[0] for r in rows]
        self.cats = [r[1] for r in rows]
        self.E = np.stack([np.frombuffer(r[2],
                dtype=np.float32) for r in rows]) if rows else np.zeros((0, 512), np.float32)
        self.threshold = threshold

    def match(self, emb):
        if len(self.E) == 0:
            return "unknown", "unknown", 0.0
        sims = self.E @ emb                        # cosine similarity (embeddings are normalised)
        # average of the top-3 similarities per person is more stable than a single best match
        best_name, best_score = "unknown", -1.0
        for name in set(self.names):
            idx = [i for i, n in enumerate(self.names) if n == name]
            score = float(np.sort(sims[idx])[-3:].mean())
            if score > best_score:
                best_name, best_score = name, score
        if best_score >= self.threshold:
            return best_name, self.cats[self.names.index(best_name)], best_score
        return "unknown", "unknown", best_score
```

9.1 The two errors you must understand

Error	Meaning	Consequence	Controlled by
False accept (FAR)	A stranger is recognised as a known person	Dangerous: no alert for a stranger	Raise the threshold to reduce it
False reject (FRR)	A known person is called unknown	Annoying: extra alerts	Lower the threshold to reduce it

For a security system, a false accept is worse than a false reject. Start with threshold 0.45 to 0.50, then choose the final value from your own evaluation in chapter 17. Never trust a default number; measure.

What you do now: Test the matcher offline with photos not used in enrollment. Print name and score for each. Also test with 5 photos of strangers: they must come out 'unknown'.

10. Tracking and multi-frame voting

A single frame can be wrong (blur, odd angle). A real system watches the same person across many frames and decides by **voting**. This needs a tracker that gives the same ID to the same person from frame to frame. A simple centroid tracker is enough for a doorway and shows that you understand the concept.

```
# src/tracker.py
import math

class CentroidTracker:
    def __init__(self, max_dist=90, max_missing=8):
        self.next_id, self.objs = 0, {}
        self.max_dist, self.max_missing = max_dist, max_missing

    def update(self, boxes):
        results, used = [], set()
        for b in boxes:
            c = ((b[0] + b[2]) // 2, (b[1] + b[3]) // 2)
            best, bd = None, self.max_dist
            for oid, o in self.objs.items():
                if oid in used:
                    continue
                d = math.dist(c, o["c"])
                if d < bd:
                    best, bd = oid, d
            if best is None:
                best = self.next_id
                self.next_id += 1
                self.objs[best] = {"c": c, "prev": c, "missing": 0}
            else:
                o = self.objs[best]
                o["prev"], o["c"], o["missing"] = o["c"], c, 0
            used.add(best)
            results.append((best, b))
        for oid in list(self.objs):
            if oid not in used:
                self.objs[oid]["missing"] += 1
                if self.objs[oid]["missing"] > self.max_missing:
                    del self.objs[oid]
        return results

# add to src/matcher.py
from collections import defaultdict, deque, Counter

class IdentityVoter:
    def __init__(self, window=8, votes_needed=3):
        self.hist = defaultdict(lambda: deque(maxlen=window))
        self.votes_needed = votes_needed

    def add(self, key, name):
        self.hist[key].append(name)

    def decide(self, key):
```

```
        # unknown votes do not cancel a good match
        names = [n for n in self.hist[key] if n != "unknown"]
        if not names:
            return None
        name, count = Counter(names).most_common(1)[0]
        return name if count >= self.votes_needed else None
```

Note: Good to know: with 'unknown' votes ignored, you only decide a name when it was seen at least 3 times. A person who never gets 3 good frames within 6 seconds is reported as 'unknown / face not clear' - see chapter 12.

What you do now: Write a test with fake detections moving across the screen and confirm the tracker keeps the same ID. Later, in the real pipeline, draw the ID next to each box.

11. Recognising the family cars

You have two realistic options. Do option A first. Option B is a good extension that shows creativity.

Option A - License plate recognition (reliable)

1. Detect the vehicle with YOLO (done in chapter 6).
2. Crop the lower half of the vehicle box (where the plate is) and upscale it 2x.
3. Run OCR on the crop, clean the text, compare with the plates in the database (allow 1 wrong character).
4. Vote across several frames, same as faces.

```
# src/plates.py
import re, easyocr, cv2
from difflib import SequenceMatcher

class PlateReader:
    def __init__(self, langs=("en",)):          # for Arabic plates use ("ar", "en")
        self.reader = easyocr.Reader(list(langs), gpu=False)

    def read(self, frame, vbox):
        x1, y1, x2, y2 = vbox
        h = y2 - y1
        crop = frame[y1 + h // 2:y2, x1:x2]       # lower half of the vehicle
        if crop.size == 0:
            return None
        crop = cv2.resize(crop, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)
        results = self.reader.readtext(crop)
        best = None
        for _, text, conf in results:
            t = re.sub(r"[^0-9A-Za-z\u0600-\u06FF]", "", text).upper()
            if len(t) >= 4 and (best is None or conf > best[1]):
                best = (t, conf)
        return best                                 # (plate_text, confidence) or None

def match_plate(plate, known_plates, min_ratio=0.85):
    best, ratio_best = None, 0
    for p in known_plates:
        r = SequenceMatcher(None, plate, p).ratio()
        if r > ratio_best:
            best, ratio_best = p, r
    return best if ratio_best >= min_ratio else None
```

Add each family car: `INSERT INTO vehicles(plate, owner, category) VALUES('ABC1234','Dad car','owner');`

Note: Plate OCR needs the plate to be readable: at least about 100 pixels wide. If the car is far from the camera, use the main stream when a vehicle is detected, or place a camera closer to the driveway. Night infrared images reflect on plates; test at night.

Option B - Appearance matching (no plate needed)

- Collect 10 photos of each family car from the camera at different times.
- Compute an embedding of the vehicle crop with a pretrained CNN (for example ResNet-50 without the final layer, from torchvision).
- Compare by cosine similarity, as with faces. Add colour histogram as a second feature.
- This is a nice mini-project: report its accuracy next to plate OCR in your README.

What you do now: Save 20 vehicle crops from your recordings and test the PlateReader on them. Count how many plates are read correctly. Write that number down for your README.

12. Event logic: knocking, entering, rules

Detections are not events. A rules layer converts raw detections into meaningful events. Define these precisely:

Event	Definition	Alert?
KNOCK / VISIT	A person stays in the door zone for 3+ seconds, or the face is recognised near the door	Yes: name + photo
ENTER	A tracked person crosses the entry line from outside to inside	Yes: always
UNKNOWN	A person is present for 6+ seconds and no identity reached the vote threshold	Yes: urgent, with photo
KNOWN ARRIVAL	Owner/family recognised	Optional: quiet notification or log only
VEHICLE	Known car recognised, or unknown car stops in the driveway	Known: log. Unknown: alert

12.1 Line crossing

Draw a line across the door in the camera image (two points). A person enters when the centroid moves from one side of the line to the other. The side is the sign of a cross product:

```
def side(p, a, b):
    return (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0])

def crossed(prev, cur, a, b):
    s1, s2 = side(prev, a, b), side(cur, a, b)
    return s1 < 0 <= s2          # direction: negative side -> positive side = entering
```

To find the line coordinates, save a frame, open it in any image viewer that shows pixel positions, and note two points. If crossings are detected in the wrong direction, swap the two points.

12.2 Rules in config (src/rules.py)

```
RULES = {
    "owner":   {"alert": "quiet",   "enter": True},      # quiet = short message without sound
    "family":  {"alert": "quiet",   "enter": True},
    "dustman": {"alert": "normal",  "enter": True, "expected_days": [0, 2, 5],
            "expected_hours": (6, 11)},
    "neighbor":{"alert": "normal",  "enter": True},
    "unknown": {"alert": "urgent",  "enter": True},
}

def build_message(identity, category, kind, camera, ts):
    if kind == "unknown":
        return f"UNKNOWN person at {camera} ({ts})"
    if kind == "enter":
        return f"{identity} ({category}) ENTERED - {camera} ({ts})"
    return f"{identity} ({category}) at the door - {camera} ({ts})"

def is_unusual(category, weekday, hour):
    r = RULES.get(category, {})
    if "expected_days" in r:
        lo, hi = r["expected_hours"]
        return weekday not in r["expected_days"] or not (lo <= hour < hi)
    return False
```

The **is_unusual** function is a nice detail: if the dustman appears at 2 AM or on the wrong day, send an urgent alert even though the face is known. (Weekday numbers: Monday = 0.)

12.3 Cooldown (stops notification spam)

```
import time
class Cooldown:
    def __init__(self, seconds):
        self.seconds, self.last = seconds, {}
    def ok(self, key):
        now = time.time()
        if now - self.last.get(key, 0) >= self.seconds:
            self.last[key] = now
            return True
        return False
# usage:  if cooldown.ok((camera, identity, kind)): notify(...)
```

What you do now: Write unit tests for crossed() with 3 cases (crosses, does not cross, wrong direction) and for is_unusual(). Unit tests are visible proof of engineering quality on your GitHub.

13. Telegram notifications

13.1 Create the bot (5 minutes)

1. Open Telegram and search for @**BotFather**. Send /newbot, choose a name and a username ending in 'bot'.
2. BotFather replies with a **token** like 123456:ABC-DEF... Copy it.
3. Open your new bot and press Start, then send it any message (for example 'hi').
4. In a browser open `https://api.telegram.org/bot<TOKEN>/getUpdates` and find `"chat":{"id": 123456789`. That number is your chat id.
5. Create the file .env in door_ai with the two values (never publish it):

```
TELEGRAM_TOKEN=123456:ABC-DEF...
TELEGRAM_CHAT_ID=123456789
```

To notify the whole family, create a Telegram group, add the bot and use the group chat id (negative number).

13.2 Non-blocking sender (src/notifier.py)

Network calls are slow. Never call Telegram in your processing loop: put messages in a queue and let a background thread send them.

```
import os, queue, threading, requests
from dotenv import load_dotenv
load_dotenv()

TOKEN = os.getenv("TELEGRAM_TOKEN")
CHAT = os.getenv("TELEGRAM_CHAT_ID")
q = queue.Queue(maxsize=50)

def _worker():
    while True:
        text, image_path, silent = q.get()
        try:
            base = f"https://api.telegram.org/bot{TOKEN}"
            if image_path:
                with open(image_path, "rb") as f:
                    requests.post(f"{base}/sendPhoto", timeout=20,
                                  data={"chat_id": CHAT, "caption": text,
                                        "disable_notification": silent},
                                  files={"photo": f})
            else:
                requests.post(f"{base}/sendMessage", timeout=20,
                              data={"chat_id": CHAT, "text": text, "disable_notification": silent})
        except Exception as e:
            print("telegram error:", e)       # the pipeline must never crash because of the network
        q.task_done()

threading.Thread(target=_worker, daemon=True).start()

def notify(text, image_path=None, silent=False):
    try:
        q.put_nowait((text, image_path, silent))
    except queue.Full:
        pass
```

What you do now: Run python -c "from notifier import notify; notify('test from door_ai')" and check your phone. Then send a test photo. Both must arrive within 5 seconds.

Note: Telegram needs internet. Add a small offline queue (save pending alerts to a file) as an extension, and an 'I am alive' heartbeat message every morning so you know the system is still running.

14. The main pipeline (putting it all together)

This is the complete flow for one frame of one camera: motion gate, vehicle and person detection, tracking, face recognition on a person crop, voting, event logic, snapshot, database and notification. Study it, then rewrite it in your own style and split it into functions with tests.

```
# src/main.py
import time, os, cv2, yaml
import database as db
from camera import CameraStream
from motion import MotionDetector
from detector import ObjectDetector
from faces import FaceEngine
from matcher import FaceMatcher, IdentityVoter
from tracker import CentroidTracker
from rules import RULES, build_message, is_unusual
from notifier import notify
import datetime

cfg = yaml.safe_load(open("config.yaml"))
P = cfg["processing"]
con = db.connect()
detector = ObjectDetector(size=P["yolo_size"])
```

```
faces = FaceEngine()
matcher = FaceMatcher(con, threshold=P["face_threshold"])
voter = IdentityVoter(votes_needed=P["votes_needed"])
os.makedirs("snapshots", exist_ok=True)

class Cooldown:
    def __init__(self, s): self.s, self.last = s, {}
    def ok(self, k):
        n = time.time()
        if n - self.last.get(k, 0) >= self.s:
            self.last[k] = n
            return True
        return False
cooldown = Cooldown(P["alert_cooldown_seconds"])

def side(p, a, b):
    return (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0])

def snapshot(frame, box, tag):
    x1, y1, x2, y2 = box
    img = frame.copy()
    cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
    path = f"snapshots/{datetime.datetime.now():%Y%m%d_%H%M%S}_{tag}.jpg"
    cv2.imwrite(path, img)
    return path

def raise_event(cam, kind, identity, category, score, frame, box):
    key = (cam, identity, kind)
    if not cooldown.ok(key):
        return
    ts = f"{datetime.datetime.now():%H:%M:%S}"
    path = snapshot(frame, box, kind)
    db.log_event(con, cam, kind, identity, category, score, path)
    rule = RULES.get(category, RULES["unknown"])
    now = datetime.datetime.now()
    text = build_message(identity, category, kind, cam, ts)
    urgent = kind == "unknown" or is_unusual(category, now.weekday(), now.hour)
    if urgent:
        text = "ALERT: " + text
    if urgent or rule["alert"] != "quiet" or kind == "enter":
        notify(text, path, silent=(rule["alert"] == "quiet" and not urgent))

def run():
    cams = {c["name"]: CameraStream(c["name"], c["rtsp_sub"]) for c in cfg["cameras"]}
    state = {c["name"]: {"motion": MotionDetector(), "tracker": CentroidTracker(),
                         "first_seen": {}, "decided": {}, "last_motion": 0,
                         "line": c["entry_line"]} for c in cfg["cameras"]}
    interval = 1.0 / P["fps"]
    while True:
        t0 = time.time()
        for name, cam in cams.items():
            frame = cam.read()
            if frame is None:
                continue
            st = state[name]
            if st["motion"].detect(frame):
                st["last_motion"] = time.time()
            if time.time() - st["last_motion"] > 4:        # no motion for 4 s: sleep this camera
                st["tracker"].update([])
                continue
            dets = detector.detect(frame)
            tracked = st["tracker"].update([d["box"] for d in detector.persons(dets)])
            for tid, box in tracked:
                key = (name, tid)
                st["first_seen"].setdefault(tid, time.time())
                x1, y1, x2, y2 = box
                h = y2 - y1
                # head and shoulders
                crop = frame[max(0, y1 - 10):y1 + int(h * 0.6), max(0, x1 - 10):x2 + 10]
                if crop.size and tid not in st["decided"]:
                    for f in faces.analyse(crop):
                        if f["size"] >= 50 and f["det_score"] >= 0.6:
                            who, cat, score = matcher.match(f["embedding"])
                            voter.add(key, who)
                    who = voter.decide(key)
                    if who:
                        cat = matcher.cats[matcher.names.index(who)]
```

```
                    st["decided"][tid] = (who, cat)
                    raise_event(name, "visit", who, cat, 1.0, frame, box)
                elif time.time() - st["first_seen"][tid] > P["unknown_after_seconds"]:
                    st["decided"][tid] = ("unknown", "unknown")
                    raise_event(name, "unknown", "unknown", "unknown", 0.0, frame, box)
            # entry-line crossing
            o = st["tracker"].objs.get(tid)
            a, b = st["line"]
            if o and side(o["prev"], a, b) < 0 <= side(o["c"], a, b) and tid in st["decided"]:
                who, cat = st["decided"][tid]
                raise_event(name, "enter", who, cat, 1.0, frame, box)
        time.sleep(max(0, interval - (time.time() - t0)))

if __name__ == "__main__":
    run()
```

Note: This code is intentionally compact. Your engineering upgrade tasks: split it into classes, add logging instead of print, handle camera loss with a 'camera offline' alert, add vehicle handling from chapter 11, and write tests for the rule logic.

What you do now: Run python src/main.py with a display-free setup. Walk to the door: you should get a Telegram message with your name. Ask a friend who is not enrolled to do the same: you should get an UNKNOWN alert after about 6 seconds.

15. Dashboard with FastAPI

A small web page with the latest events makes your demo and README look professional. Start simple: a JSON API plus an HTML table.

```
# src/dashboard.py        run: uvicorn dashboard:app --host 0.0.0.0 --port 8000
from fastapi import FastAPI
from fastapi.responses import HTMLResponse, FileResponse
import database as db

app = FastAPI()
con = db.connect()

@app.get("/api/events")
def events(limit: int = 50):
    rows = con.execute("SELECT ts,camera,kind,identity,category,score,snapshot FROM events "
                       "ORDER BY id DESC LIMIT ?", (limit,)).fetchall()
    return [dict(zip(["ts", "camera", "kind", "identity", "category", "score", "snapshot"],
            r)) for r in rows]

@app.get("/snap")
def snap(path: str):
    if not path.startswith("snapshots/") or ".." in path:      # basic path-traversal protection
        return {"error": "forbidden"}
    return FileResponse(path)

@app.get("/", response_class=HTMLResponse)
def home():
    rows = events(30)
    tr = "".join(f"<tr><td>{r['ts']}</td><td>{r['camera']}</td><td>{r['kind']}</td>"
                 f"<td>{r['identity']}</td>"
                 f"<td><a href='/snap?path={r['snapshot']}'>photo</a></td></tr>" for r in rows)
    head = "<tr><th>Time</th><th>Camera</th><th>Event</th><th>Who</th><th>Image</th></tr>"
    return f"<h2>Door events</h2><table border=1 cellpadding=6>{head}{tr}</table>"
```

- Extensions: filter by date and person, show a timeline chart (events per hour), an 'enroll new person' upload page, camera status (online/offline).
- Security: the dashboard is only for your home network. If you expose it to the internet, put it behind a VPN such as WireGuard or Tailscale, never an open port.

16. Performance tuning for an old laptop

Technique	How	Typical gain
Use sub streams for AI	640x360 or 640x480 instead of 1080p	3x to 6x faster
Motion gating	AI runs only when something moves	80-95% less CPU at idle
Lower FPS	3-5 FPS is enough for a doorway	2x to 5x
Smaller YOLO input	imgsz 320-480	1.5x to 3x
Export YOLO to ONNX	model.export(format='onnx')	1.5x to 2x
Face on the person crop only	Do not scan the full frame	2x to 4x
Skip faces after a decision	Once a track is identified, stop embedding it	Large
Small face pack	buffalo_sc instead of buffalo_l	2x to 3x
Keep recording separate	FFmpeg -c copy needs almost no CPU	Avoids overload

How to measure (always measure before optimising)

```
import time
t = time.perf_counter()
dets = detector.detect(frame)
print("yolo ms:", (time.perf_counter() - t) * 1000)
# do the same for motion, faces.analyse, matcher.match and print a table
```

- Monitor the laptop with `htop` and keep CPU average under about 70%, with temperature in a safe range. Clean the fans and put the laptop on a hard surface.
- Disable sleep and screen blanking: Settings > Power > never suspend, and disable closing-the-lid suspend (logind.conf: HandleLidSwitch=ignore).
- If the laptop is still too slow, process one camera per second in turns (round robin), or add a second cheap device like a Raspberry Pi 5 or a USB Coral accelerator later.

17. Evaluating accuracy like an engineer

This chapter turns your project from 'it works on my machine' into evidence. It is the part recruiters remember.

17.1 Build the test set

1. Collect **new** images that were never used in enrollment: at least 20 per person, from the real cameras, in different light and time of day.
2. Add **20 to 50 images of strangers** (volunteers, public face datasets such as LFW for the 'different people' pairs).
3. Label each image with the true identity in a CSV: path,identity (identity = 'unknown' for strangers).
4. Never tune on the same images that you report results on. Split into a tuning set (for the threshold) and a final test set.

17.2 Compute the metrics

```
# eval/eval_faces.py
import numpy as np, csv, cv2
from sklearn.metrics import roc_curve
from faces import FaceEngine
import database as db
from matcher import FaceMatcher

con = db.connect()
engine = FaceEngine(det_size=(640, 640))
m = FaceMatcher(con, threshold=0.0)            # threshold 0: we only want raw scores

scores, labels = [], []                         # label 1 = the best match is the correct identity
for path, truth in csv.reader(open("eval/test.csv")):
    img = cv2.imread(path)
    fs = engine.analyse(img)
    if len(fs) != 1:
        continue
    name, cat, score = m.match(fs[0]["embedding"])
    scores.append(score)
    labels.append(1 if (name == truth) else 0)

fpr, tpr, thr = roc_curve(labels, scores)
for t in np.arange(0.30, 0.71, 0.05):
    pred = np.array(scores) >= t
    lab = np.array(labels) == 1
    far = np.mean(pred[~lab]) if (~lab).any() else 0     # false accepts among wrong/unknown
    frr = np.mean(~pred[lab]) if lab.any() else 0        # known people rejected
    print(f"threshold {t:.2f}  FAR {far:.3f}  FRR {frr:.3f}")
```

- Choose the threshold where FAR is below about 1%, then report the FRR you get at that point.
- Also report the **end-to-end** result: of 50 real door visits, how many were identified correctly, how many were unknown, and how many were wrong.
- Report latency (milliseconds from frame to notification) and CPU use. Put these numbers in a table in the README.

17.3 Typical problems and what to try

- Many known people rejected at night: add infrared enrollment photos, lower the threshold slightly, enable image enhancement (CLAHE on the face crop).
- Similar-looking family members confused: add more photos for both, raise the threshold, require 4 votes for them.
- Strangers accepted: raise threshold, require a minimum face size, use the top-3 average (already in the matcher).

What you do now: Create eval/test.csv with at least 60 labelled images, run the script, choose your threshold, and save the result table as eval/results.md.

18. Running 24/7 as a service

18.1 systemd services (Ubuntu)

Create /etc/systemd/system/door-ai.service (adjust the paths and user):

```
[Unit]
Description=Door AI pipeline
After=network-online.target
Wants=network-online.target

[Service]
User=youruser
WorkingDirectory=/home/youruser/door_ai
ExecStart=/home/youruser/door_ai/.venv/bin/python src/main.py
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Do the same for recorder.py (door-recorder.service) and the dashboard (uvicorn command). Then:

```
sudo systemctl daemon-reload
sudo systemctl enable --now door-recorder door-ai door-dashboard
journalctl -u door-ai -f          # live logs
```

18.2 Health checks

- Heartbeat: send 'system alive' to Telegram every morning and when the service restarts.
- If a camera has no frames for 60 seconds (use CameraStream.is_alive()), send 'Camera offline'.
- Check free disk space daily and alert when it is below 10%.
- Set the BIOS option 'power on after power loss' so the laptop restarts after a power cut.

18.3 Docker (optional, good for your CV)

Write a Dockerfile with python:3.11-slim, install ffmpeg and requirements, and a docker-compose.yml with three services (recorder, ai, dashboard) sharing the recordings, snapshots and db folders as volumes. It proves you can ship software, not only train models.

19. Packaging the project for your CV

19.1 GitHub repository checklist

- README with: problem, architecture diagram, demo GIF or video, install steps, config explanation, results table, limitations, privacy section.
- Clean code: type hints, docstrings, logging, no secrets (use .env.example).
- tests/ with pytest for line crossing, rules, matcher and cooldown. docs/DATA_CARD.md and docs/MODEL_CARD.md from Part 2.
- eval/ folder with scripts and the final metrics.
- A short blog post or LinkedIn post that explains what you learned. Blur real faces in any public demo.

19.2 CV bullet examples (replace the numbers with yours)

- Built an end-to-end real-time home security system (Python, OpenCV, YOLOv8, ArcFace) processing 2 RTSP camera streams on CPU-only hardware at 4 FPS.
- Designed a cascade (motion gate, object detection, face embedding, multi-frame voting) that cut CPU use by about 85% and reduced false alerts with an unknown-person policy.
- Built a custom face dataset from months of security footage (semi-automatic labelling with embedding clustering, video-level splits) and trained an ArcFace-loss adapter in PyTorch, cutting false rejects from X% to Y% at a 1% false-accept rate versus a prototype baseline.
- Reached 1% false-accept rate and 94% true-accept rate on a held-out test set; threshold chosen on validation from open-set metrics.
- Delivered real-time Telegram alerts with snapshots, SQLite event logging, a FastAPI dashboard, systemd services and automated retention.

19.3 Interview questions you will be asked

- Why cosine similarity and normalised embeddings?
- What is the trade-off between FAR and FRR, and how did you choose the threshold?
- How do you handle low light, masks, glasses and new people?
- What happens when the network or a camera fails?
- How would you scale this to 20 cameras and a GPU server?
- What are the privacy and spoofing risks (a photo held in front of the camera)?

Note: Spoofing: a printed photo can fool basic face recognition. Mention it honestly as a limitation, and optionally add a liveness check (blink or motion consistency) as future work.

20. 11-week schedule

Week	Goal	Finished when
1	Setup, install, cameras connect (ch. 2-3), FFmpeg recording and cleanup (ch. 4)	2 live windows and recordings saved
2	Motion and YOLO detection (ch. 5-6)	Boxes on people and cars in real time
3	Face engine, database, photo enrollment (ch. 7-8)	Baseline recognition works on photos
4	Old footage: inventory, segments, face crops (T1-T3)	Thousands of aligned crops extracted
5	Clustering, labelling, splits, data card (T4-T5)	Labelled dataset and splits.csv ready
6	Training A and B, experiments (T6-T7)	Models beat the baseline on validation
7	Matcher, tracker, voting, rules, Telegram (ch. 9-13)	Phone alerts on knock, enter and unknown
8	Main pipeline, cars: vehicle training (ch. 14, 11, T9)	Runs 24 hours without crashing
9	Final test evaluation, failure analysis, integrate trained model (T10, ch. 17)	Results table and threshold chosen
10	Dashboard, services, active-learning loop, optional T8 (ch. 15, 18, T10.5)	System runs as a service
11	Docker, README, data and model cards, demo video (ch. 19)	Public GitHub repo ready

21. Troubleshooting table

Problem	Likely cause	Fix
VideoCapture cannot open the URL	Wrong path, password characters, camera blocks too many connections	Test in VLC, URL-encode the password, use another user, close the phone app
Video is delayed by many seconds	Frames buffered	Use the threaded CameraStream, buffer size 1, process fewer frames
Green or corrupted frames	UDP packet loss	Use TCP: OPENCV_FFMPEG_CAPTURE_OPTIONS='rtsp_transport;tcp' before starting Python
Known person shown as unknown	Face too small, blurry, dark, few enrollment photos	Add photos from the door camera, raise camera resolution, lower threshold slightly
Stranger shown as a family member	Threshold too low or poor enrollment	Raise threshold, re-evaluate FAR, add 'second best' margin check
Too many alerts	No cooldown, motion too sensitive	Cooldown per identity, higher min_area, quiet mode for family
CPU 100%, laptop hot	AI running continuously	Motion gate, 3 FPS, smaller imgsz, clean fans
No Telegram messages	Wrong chat id, bot not started, no internet	Message the bot first, recheck getUpdates, test with curl
Night images unusable	Infrared glare, noise	Enroll IR photos, CLAHE enhancement, add light near the door

A useful trick for the TCP setting: put `os.environ['OPENCV_FFMPEG_CAPTURE_OPTIONS'] = 'rtsp_transport;tcp'` at the very top of camera.py, before importing cv2.

22. Resources and books

Books

- **Deep Learning for Computer Vision with Python** (Adrian Rosebrock) - practical face recognition, object detection, deployment.
- **Programming Computer Vision with Python** (Jan Erik Solem) - image fundamentals.
- **Learning OpenCV 4 Computer Vision with Python 3** (Joseph Howse, Joe Minichino) - OpenCV, video, motion detection.
- **Practical Deep Learning for Cloud, Mobile and Edge** (Anirudh Koul et al.) - running models on small hardware.
- **Fluent Python** (Luciano Ramalho) and **Architecture Patterns with Python** - to write clean, professional code.

Documentation and papers

- OpenCV documentation (docs.opencv.org), FFmpeg documentation (ffmpeg.org/documentation.html).
- InsightFace project (github.com/deepinsight/insightface) and the paper: ArcFace: Additive Angular Margin Loss for Deep Face Recognition.
- Ultralytics YOLO documentation (docs.ultralytics.com).
- SCRFD face detector paper: Sample and Computation Redistribution for Efficient Face Detection.
- Labeled Faces in the Wild (LFW) dataset for testing verification.

Your next action

What you do now: Start with chapters 2 and 3 today. When your two camera windows are live, come back and tell me the camera brand and the result of the RTSP test, and I will help you with the exact URL, the entry-line coordinates, and any error you meet.